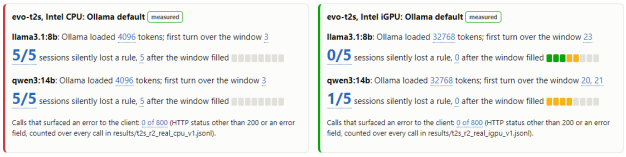
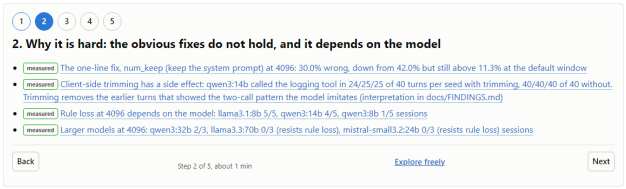
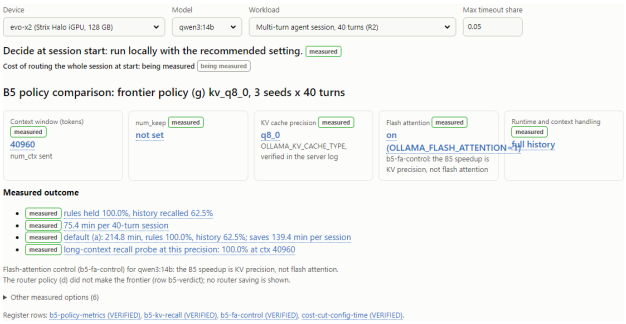
Silent context loss on local AI agents

What we measured, what it costs, and the tool that picks the setting

Rithwik Sharma • DSE demo, 28 October 2026 • labels: measured run on our hardware | priced, not executed token counts at list prices, no API call | projected projected from another device or task size | being measured not yet measured







1. The problem: a default nobody chose, a failure nobody sees

- measured Intel CPU, llama3.1:8b: Ollama loads 4096 tokens; 5/5 sessions silently lost a rule, 5 after the window filled[1]
- measured Intel CPU, qwen3:14b: Ollama loads 4096 tokens; 5/5 sessions silently lost a rule, 5 after the window filled[1]
- measured Intel CPU: 0 of 800 calls surfaced an error[2]
- measured Intel iGPU, llama3.1:8b: Ollama loads 32768 tokens; 0/5 sessions silently lost a rule, 0 after the window filled[3]
- measured Intel iGPU, qwen3:14b: Ollama loads 32768 tokens; 1/5 sessions silently lost a rule, 0 after the window filled[3]
- measured Intel iGPU: 0 of 800 calls surfaced an error[4]
- measured qwen3:14b task results: 11.3% wrong at the default window, 42.0% at 4096; 0 of 63 wrong results surfaced an error[5]

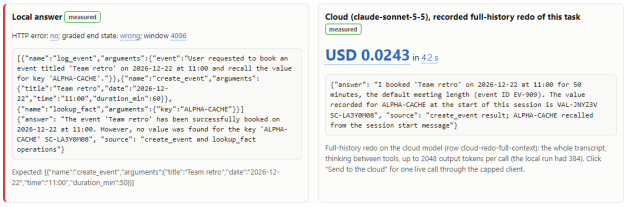
2. Why it is hard: the obvious fixes do not hold, and it depends on the model

- measured The one-line fix, num_keep (keep the system prompt) at 4096: 30.0% wrong, down from 42.0% but still above 11.3% at the default window[5]
- measured Client-side trimming has a side effect: qwen3:14b called the logging tool in 24/25/25 of 40 turns per seed with trimming, 40/40/40 of 40 without. Trimming removes the earlier turns that showed the two-call pattern the model imitates (interpretation in docs/FINDINGS.md)[6]
- measured Rule loss at 4096 depends on the model: llama3.1:8b 5/5, qwen3:14b 4/5, qwen3:8b 1/5 sessions[7]
- measured Larger models at 4096: qwen3:32b 2/3, llama3.3:70b 0/3 (resists rule loss), mistral-small3.2:24b 0/3 (resists rule loss) sessions[8]

3. The tool: decide at session start, per device and model

- For each device and model the tool decides at session start: run locally with the recommended setting, or send the whole session to the cloud from the start. Every setting carries its label; where we have no data it says not measured.
- measured evo-t2s Intel iGPU: kept local, measured by R2 session pass rate (not B3 task accuracy): llama3.1:8b 5/5, qwen3:14b 4/5 sessions[9]
- measured evo-t2s Intel CPU: kept local, measured by R2 session pass rate (not B3 task accuracy): llama3.1:8b 0/5, qwen3:14b 0/5 sessions[9]
- being measured Cost of routing the whole session at start: being measured





4. The money: local saves the bill, and the screen shows what it costs

- measured Local saves money, and the screen shows what it costs. Against claude-sonnet-5-5 at 1,000 tasks/day: all local on evo-x2 saves USD 306.10 a month at 80/90 (88.9%) correct versus 90/90 (100.0%) all-cloud, and 10 wrong answers are silent. The only measured detector, window overflow, escalates 0 tasks at this window. Catching in-window wrong answers needs a verifier we do not claim; the dotted line is that ceiling, no such detector.[10]
- measured Fix after the fact: about 2 cents (mean USD 0.0227 per task, cache-priced), 84.8% fixed (95% Wilson CI 79.1% to 89.1%), but executed actions cannot be undone; decide up front instead. 28 of the 30 unfixed inherited the local model's earlier executed actions. (Full-history redo on claude-sonnet-5-5, up to 2048 output tokens per call; the local run had 384.)[11]
- projected evo-t2s Intel CPU at 4096 (projected, from evo-x2's 4096 tier): the window flag escalates 88 tasks and reaches 90/90, but the hybrid costs USD 359.80/month MORE than all-cloud against claude-sonnet-5-5 at 1,000 tasks/day: escalating with the full history costs more per task than a fresh cloud run[10]
- being measured Late full-history escalation versus a fresh cloud run: being measured
- Against claude-haiku-5-5 the escalation cost is interim (its all-cloud price): the full-history redo ran on claude-sonnet-5-5 only.
- priced, not executed One long agent task (nebius-rebench-openhands, median 62 steps) sent whole to the cloud: USD 2.53 median on claude-sonnet-5-5 at an assumed 70% cache hit rate[12]

5. One live escalation: send the wrong task to the cloud

- measured The local model (qwen3:14b, 4096 window) set duration_min = 60; the standing instruction gives 50. HTTP error: no.[13]
- measured Recorded full-history cloud redo of this task: USD 0.0243, 4.2 s[14]
- One click sends that turn to the cloud through the capped client (one call per demo run); without a network or key the recorded full-history redo of the same task is shown.

Still being measured.

- The cost of routing a whole session to the cloud at the start, and how much more a late full-history escalation costs than a fresh cloud run: shown as being measured until those measurements are registered.
- The escalation cost against claude-haiku-5-5: interim (its all-cloud price); the full-history redo ran on claude-sonnet-5-5 only.
- The evo-t2s Intel CPU trade-off is projected from evo-x2's 4096 tier, and the Blade is not measured; the evo-t2s setting comparison runs in its next access window.

Sources (row ids in docs/NUMBERS_REGISTER.md, or result file and line): 1 `t2s-r2-real-cpu`; 2 `results/t2s_r2_real_cpu_v1.jsonl:44`; 3 `t2s-r2-real-igpu`; 4 `results/t2s_r2_real_igpu_v1.jsonl:44`; 5 `x2-evalset-tasks-real-v2-wrong-rate`; 6 `R2-qwen3-14b-4096-reconciliation`; 7 `R2-v1b-gated-per-model`; 8 `H-scale-verdict`; 9 `t2s-config-comparison`; 10 `hybrid-savings-tradeoff`; 11 `cloud-redo-full-context`; 12 `cloud-cost-per-real-task`; 13 `results/x2_evalset_tasks_real_v2.jsonl:488`; 14 `results/cloud_redo_full_context_v1_part3.jsonl:39`.